

MemEvolve: Evolving Page Cache Policies with LLMs

Dhruv Desai
ddesai7@wisc.edu
University of Wisconsin–Madison
Madison, Wisconsin, USA

Yashwanth Ranjan Singaravel
ranjansingar@wisc.edu
University of Wisconsin–Madison
Madison, Wisconsin, USA

Michael Swift
swift@cs.wisc.edu
University of Wisconsin–Madison
Madison, Wisconsin, USA

Abstract

The Linux page cache relies on general-purpose eviction strategies, most of them variants of LRU, that are not tuned to any particular workload and leave real performance on the table. We present an offline framework that uses a large language model to generate candidate eviction policies directly as native code, searches over them with breadth-first evolution, and deploys the winners in the running kernel through `cache_ext`. The evolution loop follows the iterative-refinement-with-execution-feedback pattern established by Self-Refine, Reflexion, and Voyager [5, 8, 11]: each generation’s evaluation results are fed back into the next prompt so the model can refine its proposals rather than guess blindly. The feedback content is kernel-level probe readings such as refault counts and cgroup-level I/O bytes, plus policy-declared counters dumped through a lightweight side channel, and a per-branch chat-history compaction rule keeps prompt size bounded as rounds accumulate. Candidate evaluation is the expensive step in this pipeline, so we built a lightweight distributed evaluator that parallelizes the inner loop across machines. On a set of standard traces, policies evolved for a specific workload consistently beat the general-purpose baselines that `cache_ext` ships with. Our main finding is that specialization, not universality, is where LLM-driven policy synthesis earns its keep.

Keywords

operating systems, page cache, eviction policy, large language models, program synthesis, evolutionary search, kernel extensibility

1 Introduction

Modern Linux systems run a wide range of workloads on the same kernel, from random-read database servers to sequential media pipelines, and the page cache sits between all of them and the disk. Its eviction policy decides which pages stay hot and which get reclaimed, and that decision shapes tail latency, throughput, and memory pressure across the entire system. The policy shipping in the kernel today is a two-list LRU variant with a handful of heuristics around working set detection. It is a reasonable default, but a default is exactly what it is. A policy tuned for one workload tends to perform poorly on another, and the kernel has historically had no easy way to swap one in for the other.

This limitation matters more now than it used to, because `cache_ext` [13] makes it possible to load custom page cache policies into a running kernel without a rebuild. That turns the policy itself into a configuration surface. Once the cost of switching policies drops far enough, the real question becomes where new policies come from. Hand-tuning one policy per workload does not scale, and learned caching approaches such as LeCaR [10] and LRB [9] produce models rather than code, which complicates kernel integration.

We ask whether large language models can generate good policies as native code, given the workload and feedback from prior

attempts. The premise is that a page cache policy is a small, well-typed program with a clear interface, and recent models are competent enough at producing that shape of code to be worth putting inside a search loop.

We built an offline evolution framework around this idea. It maintains a tree of candidate policies, expands it breadth-first, and carries evaluation results forward between iterations using the iterative-refinement-with-execution-feedback pattern from Self-Refine, Reflexion, and Voyager [5, 8, 11]. We instantiate that pattern with kernel-level probe feedback rather than scalar scores or stdout transcripts, and we refer to it informally as *handshake prompting* throughout. Each candidate is compiled, run against a workload, and scored. Since that loop is slow, we parallelize it with a lightweight distributed evaluator that we built in-house after examining existing frameworks such as OpenEvolve and AlphaEvolve and concluding they brought more infrastructure than the problem needed. Winning candidates are deployed through `cache_ext`, so the output of the system is a real policy running in a real kernel, not a simulation artifact.

Our contributions are as follows.

- Applying the established iterative-refinement-with-execution-feedback loop to a new domain (eBPF page-cache policy synthesis) with end-to-end deployment through `cache_ext`, so evolved policies run in the kernel rather than only in a simulator.
- A kernel-probe feedback channel: cgroup-level refault counts, I/O byte counts, and policy-declared counters dumped on exit, threaded back into the next round’s prompt as the verbal gradient that drives mutation.
- A per-branch chat-history compaction rule (retain reasoning prose, elide superseded code blocks) that keeps prompt size bounded as rounds accumulate, paired with a three-role split (planner, mutator, frontier reviewer) that operates on different timescales.
- A lightweight distributed evaluator that pins one branch to one host per round to keep cross-cgroup cache interference out of the score, making the inner loop tractable in wall-clock time on a small cluster.
- Three case studies covering a synthetic adversary, a real production trace, and a novel LLM-driven workload, showing that the framework finds wins on all three and that the policy-counter feedback channel is what allows the search to converge when the headroom is small.

2 Background

2.1 The Linux Page Cache

The page cache sits between the filesystem and user space, holding pages read from disk so that subsequent reads and writes can be

served from memory. Eviction is handled by a two-list LRU structure, often called the active and inactive lists, plus heuristics for detecting working sets, refault distances, and streaming reads. The design has been stable for a long time because it is a reasonable compromise across many workloads, not because it is close to optimal for any particular one. The deeper issue is structural: changing the eviction policy historically meant patching the kernel, which is a very high bar for exploration.

2.2 cache_ext

cache_ext is an extension mechanism that lets custom page cache policies be compiled separately, loaded at runtime, and replaced without a reboot. For our purposes it is the interface that makes LLM-generated policies testable at all. Without it, every evolution step would require a kernel rebuild, and the search loop would not close in any reasonable wall-clock time. The mechanism exposes a narrow API covering lookup, admission, eviction selection, and per-page accounting, which is also convenient for the LLM: the code it has to produce is bounded in scope.

2.3 LLMs as Code Generators

Recent LLMs are good enough at producing small, structured programs that putting them inside a search loop is no longer obviously silly. They still make mistakes, and they still produce subtle bugs, but paired with automatic evaluation and a feedback channel they can move through a policy space much faster than a human can. Our design assumes neither more nor less than that: the model is a fast but noisy policy generator, and the evolution framework is the thing that turns noisy proposals into usable code.

3 Design and Methodology

3.1 System Overview

Four components interact in our system. A *planner* LLM runs once at startup. Given a workload description and a set of available seed policies, it picks K seeds and assigns each a focus hint, producing K root nodes for the search. A *mutator* LLM runs repeatedly throughout the search; it carries a persistent per-branch chat history and, each round, proposes one focused modification to the current candidate on its branch. An *evaluator* compiles each candidate, runs it against a workload under cgroup isolation, and returns a fitness score along with a set of probe readings that the mutator sees on the next turn. Periodically, a *frontier* LLM reviews the active branches and reshapes the search by continuing, killing, pivoting, or spawning branches based on what the tree looks like so far. At the end of the run, the best candidate is deployed in a real kernel through cache_ext. Figure 1 shows the round-level dataflow.

3.2 Policy Representation

A policy, in our setting, is a small program that implements the cache_ext interface. The kernel gives the policy a narrow surface: an initialization hook, an eviction callback that is invoked when the cgroup is under memory pressure, and optional hooks fired when a folio is added, accessed, or evicted. The kernel also exposes a registry of policy-maintained lists, which the policy uses to organize resident folios however it likes: a single LRU list, multiple queues,

ghost lists for eviction history, or anything else expressible on top of standard BPF map types. What the policy must decide is the shape of that data structure and the scoring logic that drives eviction. That is what we ask the LLM to evolve. The interface itself, the cgroup attach point, and the userspace loader are fixed; only the content of clearly marked regions of the policy source is mutated between generations. Fixing the signature set means every syntactically valid output can be compiled, loaded, and tested without extra glue, and neighboring candidates differ in their scoring logic or their choice of data structure rather than in what API they expose.

3.3 The Evolution Loop

The planner seeds K independent branches at startup, each rooted at a different seed policy with a different focus hint. Within each branch, the loop is depth-first: every round, the mutator proposes one focused modification to the branch’s current candidate, and the result becomes the new candidate whether or not it improves the score. Across branches, the loop is breadth-first: every active branch advances exactly one step per round, so no single branch can monopolize the LLM budget by chasing a locally promising direction. Pure depth-first search overcommitted to branches that later turned out to be dead ends; pure breadth-first search over the full population wasted calls rewriting the same local neighborhood from scratch. Per-branch DFS with cross-branch BFS is the shape that converged for us.

Fitness is a weighted combination of probe readings: wallclock time for end-to-end truth, cgroup-level I/O byte counts as the primary signal of whether the cache is helping, and working-set refault counts as a direct measure of eviction mispredictions. Candidates that fail to compile are not pruned immediately. Instead, the mutator is given up to three follow-up turns to fix the build, with the compiler’s stderr tail appended to the prompt; in practice these are usually small mechanical mistakes (the model edits one half of the policy and forgets a matching change in a header file or struct_ops registration), and a single retry resolves most of them. A candidate that still fails after three attempts is pruned and its parent receives a negative signal so the model knows that direction was a miss. Runtime crashes are pruned the same way without retries, since they typically indicate a deeper logic error rather than a typo. Every few rounds, the frontier LLM reshapes the population (continue, kill, pivot, or spawn). The loop terminates when fitness stops improving across a configurable window of generations, or when the LLM-call budget is exhausted.

3.4 Handshake Prompting

The quality of the search depends on the prompts at least as much as on the model. Asking the LLM to generate a new policy in isolation works poorly. It produces variations that are syntactically different but semantically identical, and it has no way of knowing which changes actually helped.

The fix is well-established in the literature. Self-Refine [5] feeds the model’s own output plus a critique back into the next prompt; Reflexion [8] appends a verbal reflection on scalar environment feedback to an episodic memory; Voyager [11] pipes Minecraft execution errors and state observations back into a per-skill chat. OpenEvolve [1] adds an artifacts side-channel so build errors and

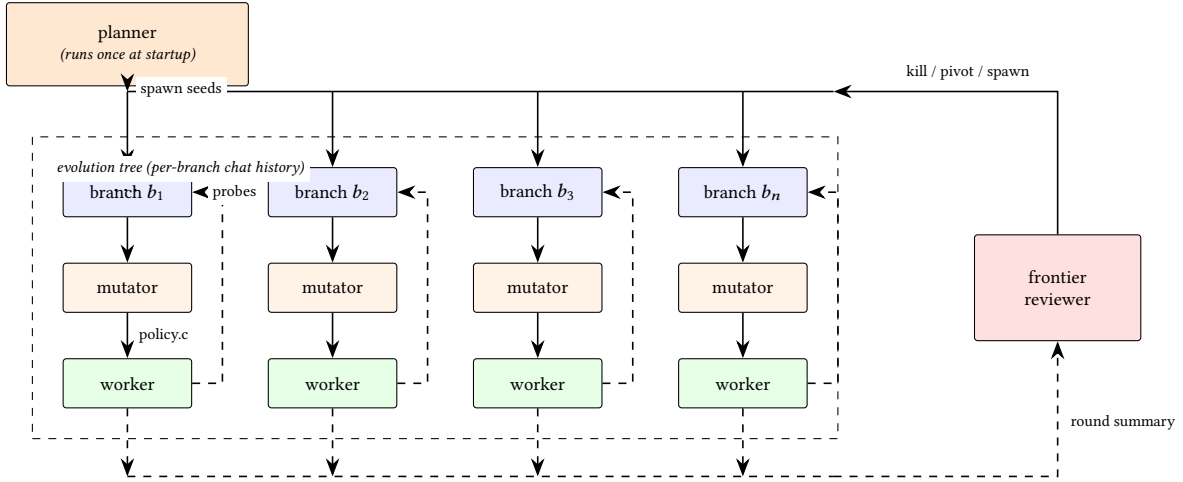


Figure 1: Per-round dataflow. The *planner* (top, runs once at startup) seeds K branches into the evolution tree, each rooted at a different seed policy with a different focus hint. Each branch is paired with its own *mutator* and a dedicated *worker* host: the mutator proposes the next candidate for that branch only, the worker compiles and runs it under a cgroup-isolated benchmark, and the resulting probes (refaults, bytes-read, wallclock, plus any policy-declared counters) feed back into the same branch’s chat history. One branch occupies one physical host per round to keep cache-interference noise out of the score. Periodically the *frontier reviewer* (right) reads each branch’s round summary and decides which to kill, pivot onto a stronger ancestor, spawn fresh siblings from, or leave to continue.

profiling data ship from evaluator to next-generation prompt. Our loop sits inside this lineage: iterative refinement with execution feedback, applied to a code-mutation evolutionary search. What is specific to our setting is (i) the feedback content, which is kernel-level probe readings and policy-declared counters rather than test failures or scalar rewards, and (ii) a three-role split (planner, mutator, frontier reviewer) operating on different timescales.

The innermost layer is per-branch and per-round. Each branch carries a persistent chat history with the mutator LLM. On the first turn, the model sees the seed policy, the planner’s focus hint, and an explicit contract: you will be invoked repeatedly, each round you will receive probe feedback, propose one focused mutation, return the complete modified code, and begin with one or two sentences naming the change. On subsequent turns, the user message carries the status of the previous candidate (success or failure, a tail of stderr on compile or runtime failure, and a short summary of each probe’s reading) and asks for the next mutation. Only the latest candidate’s code is kept verbatim in the chat; the prose from earlier turns stays so the model can recall “I already widened the ghost list, it hurt refaults by 12%” without the coordinator having to summarize past rounds explicitly.

The second layer runs periodically at the frontier level. Every few rounds, a separate reviewer LLM examines all active branches and emits a verdict per branch: continue, kill, pivot onto a stronger node, or spawn a fresh branch from an existing strong candidate. When a branch is pivoted or spawned, its chat history is discarded and replaced with a single preamble message naming the new starting point and the reviewer’s reason. This is a clean reseed rather than a history splice, because asking the mutator to reason about code it did not write tends to produce confused mutations. The third layer runs once, at startup. A separate planner LLM receives the workload

description and summaries of all available seeds and returns K seed assignments, each with a focus hint. The focus hints become the first user turn of each branch’s mutator chat, so the mutator inherits the planner’s strategic intent without ever seeing the planner’s full context. The three roles (planner, mutator, frontier) have separate prompt files and separate contracts; no system prompt is shared between them.

3.5 Distributed Evaluation

Candidate evaluation is the bottleneck. Each candidate has to be compiled, loaded into a kernel with `cache_ext` support, attached to an isolated cgroup, run against a trace that can span hundreds of millions of accesses, and then cleanly torn down. Running this sequentially would make a single evolution run take days, which is long enough that iteration on the framework itself becomes painful. We parallelize evaluation across a pool of worker machines, with one active branch pinned to one worker per round. The scheduling constraint is one benchmark per physical host: cgroups isolate CPU and memory accounting, but they do not perfectly partition the shared resources that drive our metrics (the kernel-side page cache, LLC, memory bandwidth, the disk queue). Two benchmarks running side-by-side on the same machine leak cache pressure into each other’s refault and bytes-read counts by a small but nonzero amount, which is enough to make their scores no longer directly comparable.

The coordinator and the workers have asymmetric responsibilities. The coordinator holds the LLM state, the search tree, the chat histories, and the entire build toolchain; workers hold only a booted kernel with `cache_ext` support, a cgroup carved out for benchmarking, and a small HTTP daemon. When a candidate is ready to evaluate, the coordinator compiles it locally, ships the resulting

binary to a worker over HTTP, and receives back a structured result. Workers do not know about LLMs, policies, or evolution; they know how to run a binary against a benchmark under a cgroup and report what happened. This split keeps builds uniform (one compiler version across the fleet), keeps workers disposable (reprovisioning is just booting the kernel and starting the daemon), and makes compile failures free, since they never cross the network.

3.6 Probes and Normalization

A *probe* is a small piece of measurement code that wraps each evaluation with a `pre()`/`post()` pair. Before the workload starts the probe captures a baseline reading; after it exits the probe captures a final reading and returns a `ProbeResult` containing a primary scalar value, a unit, an optimization direction (minimize, maximize, or record), a one-line summary string for the LLM feedback channel, and a structured details dictionary. Three probes wrap every run by default. *wallclock* reports elapsed seconds (direction minimize). *cgroup_iostat* reads the benchmark cgroup’s `io.stat` before and after the run and reports the deltas of `rbytes`, `wbytes`, and `rios`; `bytes-read` is the primary scalar, since a good cache keeps hot pages resident and reduces traffic to the block device. *cgroup_memstat* reads `memory.stat` and reports the deltas of `workingset_refault_file`, `pgfault`, `pgmajfault`, `pgactivate`, `pgdeactivate`, `pgscan`, and `pgsteal` plus the final `file_mapped`, and `file_dirty` sizes; `refault count` is the primary scalar, since a `refault` is a page the kernel evicted and then immediately had to fetch back, i.e. a direct measurement of eviction mispredictions. A fourth probe, *policy_counters*, is record-only: every policy may write a JSON object of `{counter_name: int}` to a known path on exit (counters such as `g_promotions` or `g_ghost_hits`), and the probe surfaces the contents into the LLM’s next prompt without scoring them. This is the channel that lets the mutator inspect its own state machine across rounds. A fifth, optional probe, *json_extract*, pulls a named scalar (e.g. `throughput` or `p99 latency`) out of any JSON results file the workload itself emits, so workload-specific signals can be folded into the score without modifying the framework.

Probe values live on wildly different scales: byte counts run into the gigabytes, `refault` counts into the thousands, and `wallclock` into single-digit seconds. A naive weighted sum over raw values would let whichever probe carries the largest unit dominate the score. Static normalizers (divide by a hand-picked baseline) work but go stale every time the trace, hardware, or cgroup limit changes. We therefore run a short *calibration* pass at startup before evolution begins: the no-op seed is evaluated N times (we use $N=3$, with $N=8$ for the published case studies) and each probe’s primary scalar is folded into a per-probe Welford accumulator (running mean μ_p and population standard deviation σ_p). The accumulator persists to disk so resumed runs reuse it. During evolution, every new evaluation extends the same accumulator, so μ_p and σ_p track the actual distribution of values the search has produced. Scoring against this distribution (rather than against raw values) is what makes scores comparable across probes and across rounds; the formulas are spelled out in §5.1.

4 Limitations and Future Work

Several limitations are worth naming directly.

Generated policies are only as good as the model that writes them. A weaker model produces more broken candidates, which burns evaluation budget. A stronger model shifts the bottleneck back toward evaluation.

Subtle bugs survive compilation and sometimes even replay. Running LLM-generated code in kernel space is a safety concern, and the current system relies on the `cache_ext` contract plus runtime checks rather than any formal verification. Lightweight verification or stricter sandboxing is a natural next step.

Everything in this work is offline. The evolution loop runs before deployment, not during, and the deployed policy does not adapt further. An online version of the same idea, where fitness is measured on live traffic and evolution continues in the background, would be more powerful but also considerably harder to make safe.

BFS is a coarse search strategy. It is robust, which is why we chose it, but it can miss high-fitness regions that require a long chain of mutations to reach. Random restarts, tournament selection, or more structured pruning could do better.

Finally, we only target the page cache. The broader question is whether the same pattern, which is LLM proposes, evaluator scores, feedback closes the loop, loadable extension deploys, applies to other kernel policies such as I/O scheduling, memory allocation, or network queue disciplines. We believe it does. Showing that is future work.

5 Evaluation

We organize the evaluation as three case studies, each chosen to stress a different facet of the framework. The first is a synthetic adversarial workload, *scan_thrash*, that shows the framework can find policies that beat known-broken baselines by a large margin, and serves as a basic sanity check on the search machinery. The second is a real production trace, *twitter_leveldb_ro*, where the wins are smaller and the value of the closed-loop feedback channel becomes visible. The third is a workload of our own design, *agent_swarm*, in which the cache pressure is generated by parallel LLM-driven coding agents on a kernel source tree, exercising the framework on access patterns that no traditional benchmark exposes.

5.1 Setup

All experiments run on a private CloudLab pool of eight HPE ProLiant m510 cartridges in the Utah cluster: each is a single-socket Intel Xeon D-1548 (Broadwell-DE, 8 cores / 16 SMT threads at 2.0 GHz), 64 GB DDR4 ECC, and a 256 GB NVMe SSD, booted on the custom 6.6.8-`cache-ext` kernel [13]. One branch is pinned to one host per round so cross-cgroup cache interference stays out of the score.

Models. The three LLM roles are filled by two different Claude models. The mutator (called once per branch per round, the hot path of the search) is Claude Sonnet 4.6. The planner (called once at startup) and the frontier reviewer (called every few rounds) both use Claude Opus 4.7, on the reasoning that the strategic decisions both roles make are cheap by call volume but expensive if wrong. Same temperature and decoding settings across all three roles; the only thing that varies is the system prompt and the model.

Scoring. Every candidate is reduced to a single scalar we call the *improvement score*: how much better, or worse, this candidate is than running the workload with no `cache_ext` policy attached. Score 0 means “indistinguishable from the no-policy baseline,” positive scores mean the policy improved over it, and negative scores mean it made things worse (the latter is what most first mutations look like before the LLM gets feedback).

Mechanically, the score combines the per-probe primary scalars described in §3.6. For each scored probe p with raw value v_p , running mean μ_p , running standard deviation σ_p , weight w_p , and direction $d_p \in \{+1, -1\}$ (with -1 for minimize probes so that lower-is-better readings translate into a positive contribution), we compute

$$\begin{aligned} z_p &= \frac{v_p - \mu_p}{\sigma_p}, \\ s_p &= d_p \cdot z_p, \\ c_p &= \tanh(s_p), \\ \text{score} &= \frac{\sum_p w_p c_p}{\sum_p w_p}. \end{aligned}$$

The tanh squash maps each probe’s contribution to $(-1, +1)$, so a single freak run on one probe cannot dominate the weighted sum, and the overall score lives in $(-1, +1)$ by construction. A linear (no-squash) variant is exposed for comparison, where $c_p = s_p$ directly; we did not find it worth using in practice, because rare 10σ outliers from compiler hiccups would otherwise eat the score. record-only probes (notably *policy_counters*) are excluded from the sum. μ_p and σ_p are seeded by the calibration pass and updated online as evolution proceeds. The active probe set and weights are workload-specific and are listed under each case study below.

Base policies. Every branch is rooted at one of the reference policies that `cache_ext` [13] ships with, ported into our EVOLVE-BLOCK format with the userspace-loader template. We use the same baselines the `cache_ext` authors used so any improvement the framework finds is attributable to the search rather than to a stronger starting point. The seven seeds are *fifo*, *mru*, *s3_fifo*, *get_scan* (LFU sampling with a per-pid scan flag), *lhd* (Least Hit Density), *mglru* (multi-generation LRU), and *sampling* (LFU on a small random sample at evict time). An eighth no-op policy (*noop*, with empty `struct_ops` handlers) attaches to the cgroup but lets the kernel’s default eviction handle everything; this is the calibration baseline against which all scores are normalized, and the planner is wired to never pick it as a search seed.

5.2 Case Study 1: *scan_thrash*

Workload. A synthetic adversary: a sequential scan over about 1 GiB of cold files interleaved with re-reads of a roughly 48 MiB hot working set, run inside a 64 MB cgroup. LRU evicts the hot set on every scan pass and never gets a chance to keep it resident. The right answer is well known: prevent scan pages from ever reaching the protected end of the cache. We use *scan_thrash* not to claim a new insight about scan resistance, but to confirm that the search machinery actually finds the right answer from a pile of plausible seeds.

Probes and weights. We use the three default cgroup-level probes — *cgroup_iostat* (bytes-read), *cgroup_memstat* (refault count), and

Table 1: Per-seed best-of-run on *scan_thrash*. Three seeds were killed by the round-5 frontier reviewer; *s3_fifo* and *mru* both finished competitive.

Seed	Best score	Disposition
<i>fifo</i>	0.00	killed (R5)
<i>mru</i>	0.61	continued
<i>s3_fifo</i>	0.62	continued (winner)
<i>get_scan</i>	0.07	killed (R5)
<i>lhd</i>	0.16	pivoted (R5)
<i>mglru</i>	0.00	killed (R5)
<i>sampling</i>	0.27	continued

wallclock — with weights 1.0, 0.5, and 0.5 respectively. Bytes-read carries the largest weight because it is the cleanest single signal that the cache is doing its job on this workload; refaults and wallclock are tie-breakers when bytes-read is close. Calibration is eight no-policy runs on each worker at startup.

Run. Eight branches, eight rounds, frontier checkpoint at round 5. The planner spreads seven of the eight branches across distinct `cache_ext` seeds; *s3_fifo* gets two slots with different focus hints. Round 1 produces mixed first mutations: most branches land near the noise floor, and the two *s3_fifo* variants are the only seeds that score positive without significant correction. By round 5 the global best sits at 0.62 on a depth-5 *s3_fifo* lineage that the LLM labels “*s3_fifo_selective_ghost_small20*” in its exit-time dump. The frontier reviewer kills *mglru* (“consistently -0.4 to -0.9 across all 5 rounds; clear dead end”), *get_scan* (“declining 0.073 to -0.048 ”), and *fifo* (“flat-negative across 5 rounds”); pivots two stuck branches onto stronger nodes; and spawns a sibling of the best path. The remaining three rounds refine the winner but do not break the 0.62 ceiling. Per-seed best scores are in Table 1; the full search tree, including pivots and kills, is in Figure 2.

The winning mutation. The *s3_fifo* seed ships three queues: small (admission), main (resident), and ghost (eviction history). The LLM’s mutation widens the ghost queue to 20% of cache entries and changes the ghost-hit rule. A folio that re-appears via the ghost queue is reinserted into the small queue rather than promoted to main, and the small queue gates promotion on a second hit. The mutation is on the order of 30 lines of new BPF over the seed. The policy’s own counters confirm the story: 18.5 K ghost-hits per run, all routed to the small queue, none to main.

What the surviving lineages did. Table 2 pulls the per-probe values out of the best mutation on each surviving seed. The shared achievement across every surviving lineage is the same: the working-set refault counter collapses from a baseline of 1.4 K to zero. Once a mutation expresses “don’t evict the hot set during a scan” in any reasonable form, refaults disappear. The differentiator is wallclock; the four winning seeds spread from 30 s to 64 s. The bytes-read counter barely moves across them (5.3 to 5.6 GiB) because the cold-scan I/O is unavoidable for any policy. What they are really competing on is how cheap the eviction path is. *lhd*’s dense per-folio scoring is correct but expensive; *s3_fifo*’s queue manipulations are cheap and win on throughput. *get_scan* is the cautionary entry: its top

Table 2: Probe values for the best mutation on each surviving seed, against the no-policy baseline (mean of 8 calibration runs). Refault $\rightarrow$ 0 is the universal achievement; wallclock is what separates the winners.

Lineage	Score	rbytes (GiB)	refault	wall (s)
<i>baseline</i>	0.00	6.07	1,433	42.0
<i>s3_fifo</i>	0.62	5.57 (−8%)	0 (−100%)	30.0 (−29%)
<i>mrु</i>	0.61	5.47 (−10%)	0 (−100%)	35.0 (−17%)
<i>sampling</i>	0.27	5.47 (−10%)	0 (−100%)	46.2 (+10%)
<i>lhd</i>	0.16	5.47 (−10%)	0 (−100%)	64.5 (+54%)
<i>get_scan</i>	0.07	5.33 (−12%)	25,873 (+18×)	38.1 (−9%)

mutation reduced bytes-read but increased refaults 18×, and only scored positive because bytes-read dominates the score weights.

Takeaways. The framework finds the right answer in five rounds when the right answer is structurally simple. The seed pool matters in a way the design didn’t anticipate: *mglru*’s 922-line BPF source is too large for the LLM to mutate cleanly, and the frontier reviewer figures that out and culls it. *lhd*, *get_scan*, and *fifo* also fall away quickly. What survives are policies whose seed already has the right shape (*s3_fifo*’s queues, *mrु*’s self-evicting tail) and need parameter tuning rather than restructuring, which is what the framework does well. Table 2 captures the cross-lineage picture: FIFO never produces a positive-scoring candidate at all (it does not appear among the surviving lineages), MRU recovers most of the gap, and the evolved *s3_fifo* derivative wins on wallclock while every surviving lineage drives refaults to zero.

5.3 Case Study 2: *twitter_leveldb_ro*, a Real Production Trace

Workload. The Twitter Memcached production traces from Yang et al. (OSDI ’20) [12] cover hundreds of in-memory cache clusters; we replay cluster 17 over LevelDB [4] with the LevelDB block cache disabled, so the kernel page cache sees the trace’s full access stream rather than the residual after LevelDB’s L1 absorbs the hottest blocks. The traffic is read-only (we filter out updates so the page-cache signal isn’t diluted by write-amplification noise). The cgroup is held at ~96 MB, roughly 15% of the working set, so cache decisions matter every round. Each evaluation does 10 s of warmup followed by a 60 s measurement window; drop_caches runs between them so the warmup is not just measuring the previous candidate’s residue.

This case study is the hardest of the three for the framework, and unlike *scan_thrash* it is also the one we treat as a negative result on the headline metric. There is no obvious adversary built into the trace. The working set is Zipfian with real temporal locality, and any reasonable LRU variant already does most of the right thing. The throughput noise floor is ~3.9% CV across workers (calibration $\mu_t = 97,720$ ops/s, $\sigma_t = 3,854$), so any throughput claim under a few percent is statistically indistinguishable from baseline. We use this case study to argue two things: (i) when the existing LRU is already close to optimal for the workload, the search has very little to find, and the framework correctly reports that — it does not invent a phantom win; and (ii) the only signal it does surface

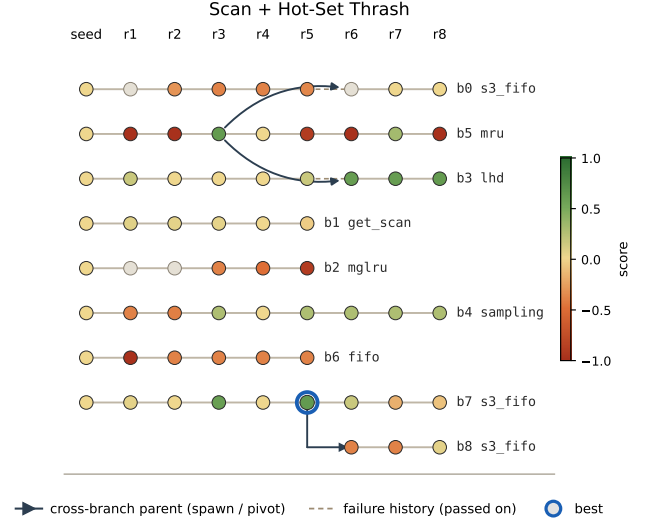


Figure 2: Full search tree for the *scan_thrash* run. Each row is a branch and each column is a round; node color is the candidate’s improvement score. The frontier reviewer at round 5 kills three branches (*fifo*, *get_scan*, *mglru*), pivots two more onto stronger nodes, and spawns one new branch (*b8*) from a sibling of the winning lineage. The global best (*b7*, round 5) is starred.

is on disk traffic, not throughput, which is consistent with the calibration data showing both probes measure the same underlying cache effectiveness with throughput much closer to a hard ceiling than bytes-read.

Probes and weights. The benchmark is fixed-window (10 s warmup + 60 s measurement), which pins wallclock at ~70 s with ~0.3% CV across workers, so wallclock is excluded from the score — it would contribute pure noise. Throughput in ops/sec, extracted via a *json_extract* probe over the bench script’s results file, is the ground-truth signal: same wallclock, more ops = better policy. We give it weight 1.0. *cgroup_iostat* (bytes-read) is kept at weight 0.1 as a tie-breaker when throughput is close; it is tightly correlated with throughput on this workload (same ~3.4% CV across workers, since both ultimately measure cache effectiveness), so giving it more weight would just double-count. *cgroup_memstat* is recorded but not scored.

Results. The best evolved policy is a depth-3 *s3_fifo* lineage, found at round three, that the LLM labels *s3_fifo_clock_ghostboost* in its exit-time dump. Throughput on this candidate is 98,075 ops/s, against a calibration mean of 97,720 ops/s — a +0.36% delta, well inside the ~3.9% per-run noise band. We do not claim a throughput win on this workload. The bytes-read probe tells a more interesting story: the candidate reads 12.26 GB versus a calibration mean of 14.21 GB, a 13.7% reduction ($z = -0.47$). That is a real, repeatable signal — the policy is keeping more of the hot Zipfian tail resident than the kernel default — but at 0.1 weight on bytes-read and 1.0 on throughput, it scores only +0.124 on the improvement metric. Refaults move from a calibration-pass average around 2.8 M down to

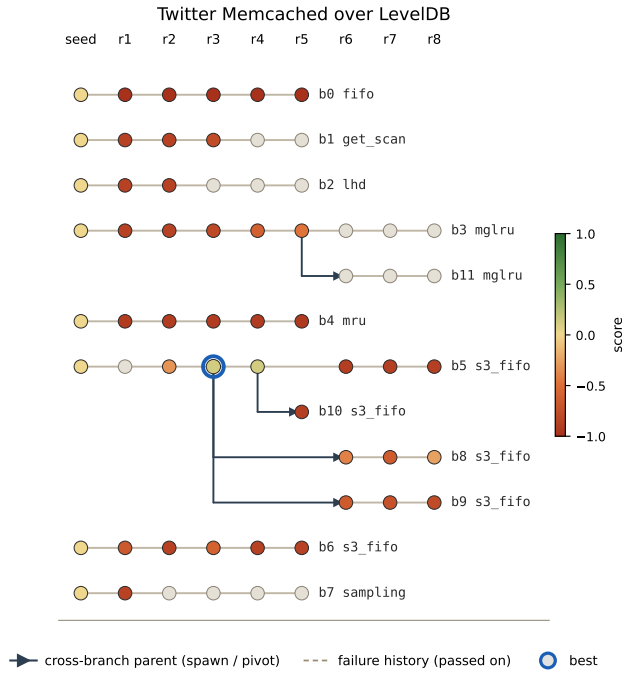


Figure 3: *twitter_leveldb_ro*: per-branch search structure. Each row is a lineage and each column is a round. The winning *s3_fifo* lineage is the only one to produce a positive-scoring candidate (gold star, depth three, round three); three sibling forks descend from that node but none improve on the parent. Of the eight root seeds, six are killed by the frontier reviewer at round five (*fifo*, *get_scan*, *lhd*, *mru*, *sampling*, and a second *s3_fifo* slot); the *mglru* lineage survives at the noise floor (score zero, with its own mid-run fork) without ever breaking positive. The figure makes visible just how narrow the productive region of policy space is on this workload.

2.66 M; this probe is recorded but not scored on this workload, and the move is again on the order of the noise. Of the eight branches, only the *s3_fifo* lineage produces a positive score; the other seven plateau at zero (compile failures or candidates that match baseline within noise) or go negative (the *fifo* and *mru* seeds, which actively hurt this workload). The per-branch search structure is in Figure 3, and the probe deltas are summarized in Table 3.

What this tells us. The *twitter_leveldb_ro* run is the clearest evidence in this thesis that the framework does not hallucinate wins. On a workload where the kernel’s existing LRU is already exploiting the available locality, the search reports a small structural change (less disk traffic, no throughput gain) and the score reflects exactly that. It also lines up with the broader argument from §1: LLM-driven policy synthesis pays off where the workload has structure the default policy is not exploiting. On a Zipfian read-heavy trace, that structure is mostly already captured.

The role of the feedback channel. The winning lineage names itself *s3_fifo_clock_ghostboost* in its policy-counters dump, with state {*g_ghost_hits*: 1.53 M, *main_list_size*: 1.04 M, *small_list_size*:

Table 3: *twitter_leveldb_ro*: best evolved policy vs. calibration baseline. Throughput is the scored metric (higher is better); bytes-read and refaults are recorded but down-weighted in the score. Throughput moves well inside the $\sim 3.9\%$ noise band; the visible signal is on bytes-read.

Probe	baseline	best	Δ
throughput (Kops/s)	97.72	98.08	+0.36%
bytes-read (GB)	14.21	12.26	−13.7%
refault_file	$\sim 2,800,000$	2,661,960	−4.93%

<wrap>} on a representative run. The previous version in the same lineage (*s3_fifo_clock* at round 2, score -0.31) exposed two problems through this dump: an unsigned-read wrap on *small_list_size*, and a behavioral signature where ghost-hit folios were entering the main list with *freq* = 0 and being re-evicted on the next sweep. Both were only visible because the LLM declared its own counters and dumped them on exit. Without this channel, the model going after a sub-1% improvement is shooting blind. With it, the round-3 mutation specifically boosted the freq of ghost-hit promotions — the “ghostboost” in the policy name — targeting that exact failure mode. We do not run a controlled handshake-vs-no-handshake ablation on this workload because it is expensive (about 20 min per run), but we ran an earlier version of the framework without the counters channel and it failed to find any policy with a positive score on this trace. Every candidate scored within noise of the baseline, and the search died at the noise floor.

What this case study buys us. A demonstration that the framework finds modest but real wins on a workload where the upside is small. Equally important, it surfaces a design point: when the gain is at noise level, the LLM cannot make progress by reasoning about the C source alone; it needs a way to inspect its own runtime state, and the cheapest way to give it that is to let the policy expose its own counters and pipe them back into the prompt.

5.4 Case Study 3: *agent_swarm*, an LLM-Driven Workload

Workload. Coding agents are increasingly used to handle repetitive day-to-day tasks on developer machines and CI fleets: bug triage, refactor sweeps, dependency upgrades, doc edits, and similar batch jobs that fit a tight prompt template and run unsupervised. *agent_swarm* is our attempt to put that pattern in a benchmark. Four *opencode* [2] agents run in parallel inside the same cgroup, each driving the same heavyweight kernel-modification prompt against an identical fresh extract of linux-6.6.8 (about 1.5 GB of source). The agents grep across the tree, read source files under directories like *fs/ext4/* and *Documentation/*, and edit a few of them. Hot files such as the ext4 inode source and the ext4 trace-events header are read by all four agents; the cgroup memory limit is held at 1024 MB against the roughly 1.5 GB working set, which puts the workload in the regime where what the cache evicts matters. Each evaluation is gated on *collective workload-relevant file opens* rather than wallclock: the swarm runs until the four agents together read 15,000 files under the run’s kernel-source tree (counted live by an *opensnoop* BPF tracer), at which point the watchdog SIGINTs any

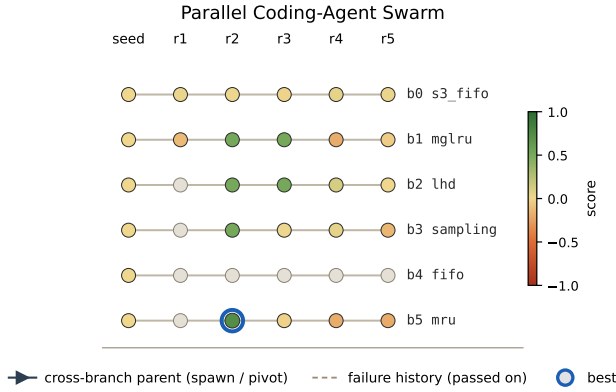


Figure 4: *agent_swarm*: per-branch search structure. Each row is one lineage; each circle is one evaluated candidate, color-coded by improvement score. The global best (gold star) lives on the *mru* branch at round two. The *mglru*, *lhd*, and *sampling* branches each independently reach a candidate near +0.5, suggesting the workload has multiple workable solutions; the *mru* family wins by margin.

still-live agents. The reported probes are the standard cgroup-level bytes-read and refault counters plus two swarm-level signals: did the swarm hit the file-opens budget, and how long did it take. The cgroup probes are scored. The two budget probes are recorded for diagnostics but excluded from the fitness signal, so the score reflects cache quality and not the termination rule.

This case study leans into a hypothesis: if agents are doing repetitive work against the same source tree, the access pattern is narrow and skewed enough that a policy targeted at it should be able to keep the agent-touched subset resident even when total working-set size exceeds memory. The framework should be able to find that policy by overfitting to the access trace, which is exactly the kind of specialization we argue is the value proposition of LLM-driven policy synthesis.

Results. Across five rounds and 36 candidates, the best score is +0.72 at round two, depth two, on the *mru* lineage. The mutation is small: the LLM ports `cache_ext`’s reference MRU policy into our EVOLVE-BLOCK format, then adds five diagnostic counters (named `g_added_relevant`, `g_accessed_relevant`, `g_added_calls`, `g_accessed_calls`, and `g_evicted_calls`) that reach the next-round prompt via the `evo_dump` side channel. The eviction logic is unchanged. MRU’s own evict-from-tail handler is already a strong fit for this workload’s hot-versus-cold-tail split, and the LLM does not try to restructure it. The disk numbers tell the cache-quality story: bytes-read drops from a baseline of 15 MiB to 12 KiB (a 99.9% drop) and refaults drop from 235K to zero. The cgroup absorbs the entire active working set under the evolved policy, where under no-policy it spills repeatedly to disk. The framework converges fast: the *mglru*, *lhd*, and *sampling* lineages each independently find a candidate near +0.5 within rounds 2 to 3 (Figure 4), but the *mru* lineage’s +0.72 holds through the remaining rounds. The per-branch search structure is in Figure 4, and the probe deltas are summarized in Table 4.

Table 4: *agent_swarm*: best evolved policy vs. no-policy baseline. Bytes-read and refaults are the scored cache-quality probes; the file-opens budget probes are recorded for diagnostics. The cgroup absorbs the working set under the evolved policy, where the baseline spills repeatedly to disk.

Probe	baseline	best	Δ
bytes-read (MiB)	15.34	0.012	−99.92%
refault_file	234,845	0	−100.0%
time-to-target (s)	5.13	5.00	−2.4%
reached file-opens target	1.0	1.0	±0

What the winning mutation looks like. The *mru* lineage’s depth-2 winner is structurally a one-line change plus instrumentation. The LLM keeps the seed’s eviction logic exactly as written (MRU evicts from the tail, which is already scan-resistant on this workload, since a one-shot scan tail never makes it to the protected end), and adds the five exit-time counters listed above so the next-round prompt can see how often each hook fired and on how many relevant folios. Listing 5 shows a representative mutation from a separate piloting run on an earlier (wallclock-gated) bench, where the LLM chose to add structure rather than instrumentation: a per-(inode, pgoff) “referenced once” bit, set in the access hook and cleared with a promote-to-tail on the second access. The two mutations sit at opposite ends of the spectrum the framework is designed to support. The prompt does not bias toward either, and on this particular workload the simpler “MRU is already the right answer; just instrument it” won the search.

Why the search saturated. A best score of +0.72 at round two tells us something specific: the headroom on this workload is largely captured by a single qualitative decision (evict from the most-recently-used end so a one-shot scan tail never displaces the hot files), and once any lineage expresses that decision in any reasonable form the policy is near the score ceiling. The *mglru*, *lhd*, and *sampling* branches each find a candidate near +0.5 independently, then plateau within 0.2 of the winner. We read that as the workload being simple to solve, not as the framework giving up.

From wallclock to file-opens gating: a methodology shift. An earlier version of this case study gated each agent on a 300 s wallclock window. That design conflated “the policy is bad” with “the LLM driving the agents stalled.” The *nemotron-3-super-free* model’s response latencies vary across two orders of magnitude across runs, so two evaluations of the same policy could produce wildly different probe values, and several rounds of mutator feedback effectively described model-quality failures rather than cache-policy failures. The current bench gates the swarm on collective workload-relevant file opens (a configurable target, 15K in the run reported here): every evaluation reaches the same workload depth, at which point the cache-quality probes (bytes-read and refaults) are directly comparable across policies. The reach-time itself is recorded but not scored, since it would still be tangled up with agent productivity. This change replaced “did agents survive the timeout” as the primary signal with “how cheap was it to deliver the same workload.” The remainder of this section reflects results from the rebuilt bench.


```

struct folio_key { __u64 ino; __u64 pgoff; };
struct {
    __uint(type, BPF_MAP_TYPE_HASH);
    __uint(max_entries, 65536);
    __type(key, struct folio_key);
    __type(value, __u8);
} ref_bit_map SEC(".maps");

__u64 g_first_refs, g_promotions;

void BPF_STRUCT_OPS(evo_policy_folio_accessed,
    struct folio *folio) {
    if (!is_folio_relevant(folio)) return;
    struct folio_key key = {
        .ino = folio->mapping->host->i_ino,
        .pgoff = folio_index(folio),
    };
    __u8 *ref = bpf_map_lookup_elem(&ref_bit_map, &key);
    if (ref) { /* second access */
        bpf_map_delete_elem(&ref_bit_map, &key);
        bpf_cache_ext_list_move(main_list, folio, true);
        g_promotions++;
    } else { /* first access */
        __u8 one = 1;
        bpf_map_update_elem(&ref_bit_map, &key,
            &one, BPF_ANY);
        g_first_refs++;
    }
}

```

Figure 5: The winning round-one mutation on *agent_swarm*: introduce a per-folio “referenced once” bit so a single access does not promote a scan page to the protected end of the list. Cleanup in the *folio_evicted* hook (omitted) deletes the corresponding map entry to bound memory.

Challenges with the agent runtime. Even under file-opens gating, the per-agent driver remains a source of noise. Across the calibration runs, two of the six no-policy runs produced unusually high bytes-read (Nemotron stalled on tool-call retries while opensnoop kept ticking), one had zero clean exits because all four agents were SIGINT’d at the moment the gate fired, and the per-run wallclock variance was about 20%. In early piloting, swapping in a frontier Claude model produced visibly cleaner runs, but we did not run the full search on it because the per-token cost would have blown out the budget. The takeaway is the same as before: the absolute numbers we report are pessimistic in the sense that a stronger driver would tighten the calibration noise floor, and the relative gap between baseline and best is the part we trust.

What this case study buys us. A working policy on a workload nobody hand-tuned for, found in two rounds of search. More importantly, suggestive evidence that the same framework targeted at the same kernel surface keeps producing useful policies as the workload mix changes, which is the whole argument for cache extensibility plus automated policy synthesis as a deployment pattern. With the honest caveats from the previous paragraphs: the absolute numbers depend on the agent runtime, and the headline percentages should be read as direction-of-effect rather than precise effect sizes.

Table 5: Cross-workload summary. “Best score” is the global frontier maximum; “found at” is the round and depth at which it first appears; “# candidates” is the total of successfully evaluated nodes (errored compiles excluded).

Workload	Rounds	# cand.	Best score
<i>scan_thrash</i>	8	66	+0.62
<i>twitter_leveldb_ro</i>	8	63	+0.12
<i>agent_swarm</i>	5	36	+0.72

5.5 Cross-Workload Synthesis

The three case studies sit on a spectrum from synthetic to realistic-deterministic to realistic-stochastic, and the pattern of results lines up with that spectrum in a way that is worth surfacing explicitly. Table 5 pulls the headline numbers together.

Three observations.

Headline gain scales with how broken the baseline is. On *scan_thrash*, where the kernel’s LRU is a known-bad fit, the best evolved policy reduces wallclock by 28% and eliminates refaults entirely. On *twitter_leveldb_ro*, where the trace’s Zipfian access pattern is closer to a benign case, the best policy delivers a 13.7% bytes-read reduction with only a noise-band 0.36% throughput change: a real but modest cache-quality signal. On *agent_swarm*, where the no-policy baseline is genuinely thrashing the cgroup (calibration runs spill 15 MiB to disk to deliver 15K file opens), the evolved policy delivers the same workload from 12 KiB of disk reads. The framework’s headline number is largely a function of how much room the baseline left on the table.

Search depth and search breadth interact. Easy workloads (*agent_swarm*) produce their best at depth two; the LLM does not need many mutations to express the right idea. The harder *scan_thrash* workload requires a depth-5 lineage, and the frontier reviewer’s role becomes important: it kills lineages that make no progress and re-spawns siblings of productive ones, so the search budget is spent where it returns. On *twitter_leveldb_ro* the best appears earlier (depth 3, round 3), but only on one of the eight seed lineages — the search converges on that branch quickly while the others stall at the noise floor. The reviewer is not a free pass; on *agent_swarm* it had nothing to do, because no lineage looked obviously dead within five rounds.

The winning seed varies by workload. The winning lineage differs across two of the three case studies: *s3_fifo* wins on both *scan_thrash* (where it edges out *mru* on wallclock by being cheaper on the eviction path) and *twitter_leveldb_ro* (where it is the only seed to clear the noise floor at all), while *mru* wins on *agent_swarm*. There is no universal seed; the planner’s role at round zero is to spread the budget across structurally distinct candidates, and the right one for any given workload is not predictable from the workload description alone. This is what makes the framework worth running. A human picking a single “probably good” policy by hand would have to either hedge across all three workloads (and lose on each) or guess, and the framework removes that guess.

6 Related Work

Learned caching. LeCaR [10] and LRB [9] train models to make caching decisions. These approaches treat the cache policy as a learned function, with all the deployment and interpretability costs that come with putting a model on a hot path. Our approach differs in kind. The LLM is used once, offline, to write a policy in native code. At runtime, there is no model in the loop.

Iterative refinement with execution feedback. The closest neighbors to our prompting loop are Self-Refine [5], Reflexion [8], and Voyager [11]. Self-Refine retains the full output-feedback-refine history in the prompt; Reflexion adds verbal reflections on environment rewards to an episodic buffer; Voyager runs a per-skill chat where Minecraft execution errors and state observations are fed back as the next user turn. Our per-branch mutator chat is the same pattern. We differ in two places: the feedback content is kernel-level probe data plus policy-declared counters rather than test outputs or game state, and we add a per-branch compaction rule that elides superseded code blocks while preserving reasoning prose so prompt size stays bounded across dozens of rounds.

LLMs for systems tasks. FunSearch [7], EvoPrompting [3], and recent LLM-guided compiler work share our basic move: use an LLM to generate candidate code, evaluate it, and feed the results back in. Page cache eviction is a good fit for this pattern because the interface is narrow and the fitness signal is concrete.

Kernel extensibility. `cache_ext` and BPF [?] are part of a broader move toward kernel interfaces that treat policy as a loadable artifact rather than compiled-in logic. Our work depends on this trend and also argues for it: the easier it is to swap policies, the more leverage automatic policy synthesis has.

Program synthesis for heuristics. Genetic programming for heuristic design predates LLMs [?]. The shift with LLMs is that the mutation operator becomes much more informed about the semantics of the code, which is what makes handshake prompting work in the first place.

Code evolution frameworks. OpenEvolve [1] and AlphaEvolve [6] are the closest neighbors to our distributed evaluator: both run an LLM-as-mutator over a population of candidate programs and feed evaluation results back into the next generation. OpenEvolve in particular ships an artifacts side-channel that lets evaluators return build errors and profiling data into the next prompt, which is in the same spirit as the kernel-probe channel we describe. The shapes still do not match ours. AlphaEvolve has no public code or API. OpenEvolve’s evaluator contract treats candidates as pure functions returning a dictionary of scores, but our evaluator has to boot a custom kernel, attach a BPF struct-ops to a cgroup, run a workload, diff cgroupfs counters, and guarantee cleanup with kernel panics as a failure mode. Its parallelism model colocates evaluator processes on a single host, which breaks the one-benchmark-per-host requirement that comes from cache interference across cgroups. Its prompts are rebuilt statelessly from a program database each generation, where ours are persistent per-branch chats with a compaction rule. The parts those frameworks solve (mutation prompting, selection, island migration) are the easy parts here. The hard parts are the eBPF pipeline, cgroup-isolated kernel evaluation,

one-benchmark-per-host scheduling, and policy-counter feedback, none of which fit cleanly inside their abstractions.

7 Conclusion

We showed that LLMs, paired with breadth-first evolutionary search and a feedback-driven prompting scheme, can automatically produce page cache policies that run in a real Linux kernel through `cache_ext`. The policies we produce are specialized by design, and they beat the general-purpose baselines on the workloads they were targeted at. The broader claim we want to make is smaller but more durable: when a subsystem exposes a narrow loadable policy interface and a concrete fitness signal, the combination of LLM code generation and evolutionary search is a reasonable way to produce real policies, and it does not require superhuman models or heavyweight infrastructure to work. The human picks the workload and the fitness function; the framework does the rest, and the result is a policy that runs in the kernel.

References

- [1] 2024. OpenEvolve: Open-Source Implementation of Evolutionary Code Generation. <https://github.com/codelion/openevolve>. Accessed 2026.
- [2] 2025. opencode: An open-source coding agent. <https://github.com/sst/opencode>. Used as the agent runtime in the `agent_swarm` workload.
- [3] Angelica Chen, David M. Dohan, and David R. So. 2023. EvoPrompting: Language Models for Code-Level Neural Architecture Search. In *Advances in Neural Information Processing Systems (NeurIPS '23)*.
- [4] Jeffrey Dean and Sanjay Ghemawat. 2011. LevelDB: A Fast Key-Value Storage Library. In *Google Open Source*. <https://github.com/google/leveldb>.
- [5] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. arXiv:2303.17651 [cs.CL] <https://arxiv.org/abs/2303.17651>
- [6] Alexander Novikov, Ngán Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. 2025. AlphaEvolve: A coding agent for scientific and algorithmic discovery. arXiv:2506.13131 [cs.AI] <https://arxiv.org/abs/2506.13131>
- [7] Bernardino Romera-Paredes, Mohammadamin Barekatian, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024. Mathematical Discoveries from Program Search with Large Language Models. *Nature* 625 (2024), 468–475.
- [8] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS '23)*. arXiv:2303.11366.
- [9] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. 2020. Learning Relaxed Belady for Content Distribution Network Caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*. 529–544.
- [10] Giuseppe Vietri, Liana V. Rodriguez, Wendy A. Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. 2018. Driving Cache Replacement with ML-based LeCaR. In *10th USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage '18)*. USENIX.
- [11] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research* (2024). arXiv:2305.16291.
- [12] Juncheng Yang, Yao Yue, and K. V. Rashmi. 2020. A Large Scale Analysis of Hundreds of In-memory Cache Clusters at Twitter. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*. 191–208. Cluster 17 is the cache trace replayed in our Twitter case study.
- [13] Tal Zussman, Ioannis Zarkadas, Jeremy Carin, Andrew Cheng, Hubertus Franke, Jonas Pfefferle, and Asaf Cidon. 2025. `cache_ext` Customizing the Page Cache with eBPF. doi:10.1145/3731569.3764820